APPLIED MACHINE LEARNING — PROJECT REPORT

Spatio-Temporal Graph Structure Learning for Traffic Speed Forecasting

Primary Paper:
Zhang et al., 201cSpatio-Temporal Graph Structure Learning for Traffic Forecasting201d
AAAI 2020, pp. 1177–1185

:
Aman Salim Mujawar
Student ID: 801498672

Course: Applied Machine Learning | Dataset: METR-LA | Framework: PyTorch

Dataset: github.com/liyaguang/DCRNN | Code: github.com/Aman-Mujawar/Spatio-Temporal-Graph

1. Introduction

1.1 Problem Statement

Urban roads keep putting out all this sensor data nonstop. Like in Los Angeles, they have 207 of those inductive loop detectors that grab vehicle speeds every five minutes, and its going on all day every day. The big problem with forecasting traffic speed is taking those recent readings from every sensor and figuring out what speeds will be like in 15, 30, or 60 minutes.

That kind of accurate prediction for short times ahead feels really crucial for smart city stuff. It helps with things like adaptive navigation apps, or changing prices for congestion on the fly, and even controlling traffic lights to respond better, plus routing for emergency vehicles. Most older methods just skip over how sensors connect in space, or they use these graphs made from actual road distances, but that might not catch the real ways drivers link them up through how they choose routes.

I think the question here gets at something specific. Can you build a model that figures out the road networks graph right from the sensor data itself, instead of locking it in based on the physical layout of roads? And would that actually lead to better forecasts for traffic speeds? It seems like it could, but Im not totally sure without testing. Some approaches try to ignore those hidden functional connections that people create just by driving patterns.

1.2 Motivation and Open Questions

There are a couple of main problems in the current research that really push this project forward. Like, a lot of the Graph Neural Network models for traffic just use road distances to set up the edges in their graphs. That works okay sometimes, but it misses stuff. For example, two roads that run parallel or ones connected by the same on-ramp might have similar traffic patterns that distance doesnt show. It seems like the graph needs to capture how traffic actually flows, not just map out the land.

That fixed setup also doesnt handle changes over time very well. Traffic shifts with seasons or big events, or when theres construction blocking things. A graph that learns from the data itself could adjust to those kinds of things, unlike sticking with some static distance thing.

This work tries to look at a few questions. One is whether learning the graph adaptively beats the usual fixed ones on those standard tests. Another, what does that learned matrix even tell us about how the roads really connect in terms of function. And then, does it keep working better for quick forecasts like 15 minutes, or even the longer 60 minute ones. I am not totally sure yet, but it feels like it might vary.

1.3 Approach Overview

I implement three models which I test against each other through their performance on the METR-LA benchmark data.

The LSTM model operates as a basic temporal model which does not include any spatial components and serves as the baseline for comparison.

The Graph WaveNet model represents the current leading method which constructs its adjacency matrix through both fixed and learnable components, while applying dilated temporal convolution operations.

Our main model STGSL learns the complete graph structure through end-to-end training of sensor embeddings without any predetermined distance information.

The research team applies the same data preparation process, training setup, and evaluation approach to all models for their performance assessment. The evaluation process uses MAE, RMSE, and MAPE metrics. These calculate values after converting predictions to miles per hour and hiding data points that are missing.

2. Background and Related Work

The table below summarises the key related methods, their approaches, strengths, and limitations relative to this project.

Model	Approach	Pros	Cons / Limitation
ARIMA/VAR	Classical statistical model; treats each sensor independently	Fast; interpretable	Cannot capture nonlinear patterns or spatial correlations
Vanilla LSTM	Deep recurrent model; flattens all sensors into one vector	Captures temporal patterns well	Completely ignores spatial structure
DCRNN (Li 2018)	GNN + diffusion convolution + seq2seq RNN; uses fixed road graph	First strong spatial-temporal model	Road graph is fixed; not learned
STGCN (Yu 2018)	GNN + gated temporal convolution; faster than DCRNN	Parallelisable; faster training	Static adjacency; cannot adapt
Graph WaveNet	Dilated TCN + GCN; adds partial adaptive adjacency term	Fast; partial adaptivity; SOTA accuracy	Adaptive part is secondary correction only
STGSL (Ours)	Fully learned graph via node embeddings $E1@E2^T$ + Dilated TCN	Graph learned end-to-end; no fixed prior needed	Needs sufficient epochs; sensitive to L1 regularisation weight

2.1 Classical Statistical Methods

ARIMA and the Kalman Filter were the standard tools for traffic forecasting for decades. They are interpretable and require minimal computation, but both assume linear relationships and treat each sensor independently. They cannot model the propagation of congestion from one road to another and break down quickly during rush-hour nonlinearities.

2.2 Deep Recurrent Models (LSTM / GRU)

Long Short-Term Memory (LSTM) networks as recurrent models enable systems to learn nonlinear time-based connections between their past data points in their own sensor data history. The process of flattening all 207 sensors into one input vector eliminates every spatial connection between the data points. The model fails to understand that sensor slowdowns start with one sensor before they spread to other sensors which are located nearby in geographic space.

Relevance to our work: LSTM is our temporal baseline. The method determines the exact value of accuracy which remains unused because spatial structure is not taken into account.

2.3 Diffusion Convolutional RNN (DCRNN — Li et al., 2018)

DCRNN was the first model to treat traffic forecasting as a signal diffusion process on a graph. The system implements pre-constructed road distance graph through bidirectional random walks which operate with an encoder-decoder recurrent network system. The research demonstrated that DCRNN outperformed LSTM baseline models while proving road graph data essential for building effective traffic prediction systems.

A diffusion model based on principles operates with a strong theoretical base from graph signal processing. The graph structure remains constant because it represents actual road distances but the model lacks ability to identify non-spatial relationships which exist between nodes. Link:

arxiv.org/abs/1707.01926

2.4 Graph WaveNet (Wu et al., 2019)

Graph WaveNet implemented Dilated Temporal Convolutional Networks (TCN) as a replacement for its recurrent temporal block which allowed simultaneous processing of time data to speed up training by three times compared to DCRNN. The system introduced two correction methods which used a fixed road-distance graph and a self-adaptive adjacency matrix based on a random learnable parameter.

Pros: The system enables rapid training and it includes some adaptive elements which achieved top accuracy on METR-LA during its initial release. Cons: The adaptive component operates as a minor element because the system continues to rely primarily on the static road network

Link:

arxiv.org/abs/1906.00121

2.5 STGSL — Our Primary Reference (Zhang et al., AAAI 2020)

STGSL makes graph learning the central, primary objective. Two trainable node embedding matrices E1 and E2 (shape N×D) replace the fixed graph entirely. The adjacency is computed as $A = \text{softmax}(\text{ReLU}(E1 @ E2^T))$ — differentiable, sparse, and requiring only 2×N×D parameters rather than N×N. L1 regularisation prevents a dense, uninformative graph. The paper reports improvements of 3–5% MAE over Graph WaveNet on METR-LA with 50+ epochs of training. Link:

ojs.aaai.org/index.php/AAAI/article/view/5470

3. Methods

3.1 Dataset — METR-LA

All experiments use the METR-LA benchmark dataset. It contains continuous speed readings from 207 inductive loop detectors on Los Angeles freeways, collected over four months (March–June 2012) at five-minute intervals. The resulting data matrix has shape (34,272 timesteps) × (207 sensors), with speed values in mph. Approximately 5–8% of values are missing and encoded as zeros. The dataset is freely available at:

github.com/liyaguang/DCRNN

3.2 Data Preprocessing

Correct preprocessing is critical for fair and meaningful comparison. The following pipeline is applied:

1. Missing value imputation. Zero-valued readings are replaced by each sensor's column mean, computed only from non-zero training values to prevent data leakage.
2. Chronological split. Data is divided 70% training / 10% validation / 20% test in strict time order — no shuffling, which would constitute temporal leakage.
3. Standardisation. A StandardScaler is fit on training data only and applied to all three splits.
4. Sliding window construction. (12-step input → 12-step output) sequence pairs are extracted, yielding ~6,000 training windows. Wrapped in PyTorch DataLoaders with batch size 64.
5. Evaluation masking. Before computing any metric, positions where the original ground truth was zero are masked out to prevent missing-value artefacts from inflating error.

3.3 Overall Architecture Framework

All three models share the same input/output contract: they receive a tensor of shape (Batch × 12 input steps × 207 sensors) and produce (Batch × 12 output steps × 207 sensors). The key difference is in how they model spatial relationships between sensors:

Input (B × 12 × 207)

↓

Temporal Block (LSTM or Dilated TCN)

↓

Spatial Block — Fixed Graph (GWN) OR Learned Graph (STGSL)

↓

Output Projection → (B × 12 × 207)

3.4 Model Architectures

Model 1 — LSTM Baseline

A two-layer LSTM with 128 hidden units. All 207 sensor readings per timestep are concatenated into a flat input vector of size 207. The final hidden state is projected by a fully connected layer to produce (12 × 207) output values. Total parameters: ~625,000. This model captures temporal dynamics but has no knowledge of spatial structure.

Model 2 — Graph WaveNet

A stack of Dilated Temporal Convolutional blocks (dilation 1→2→4, gated activation Tanh×Sigmoid) provides parallel temporal processing. Graph convolution is then applied using the formula H_out = (A_fixed + A_adaptive) × H_in × W. A_fixed is the normalised road-distance adjacency; A_adaptive is a trainable random matrix. The combination allows the model to correct the pre-built graph with data-driven edges.

Model 3 — STGSL (Primary)

The adaptive adjacency formula is the core innovation: $A = \text{softmax}(\text{ReLU}(E1 @ E2^T))$ where $E1, E2 \in \mathbb{R}^{(207\times10)}$ are trainable node embedding matrices. This produces a 207×207 adjacency with only 2×207×10 = 4,140 parameters — ten times fewer than a direct N×N matrix. L1 regularisation on A enforces sparsity. The temporal block is identical to Graph WaveNet (Dilated TCN), ensuring the spatial components are the only variable between the two models.

3.5 Training Configuration

All three models are trained under identical settings:

- Optimiser: Adam — learning rate 1e-3, weight decay 1e-4
- Loss: Huber Loss (smooth L1) — robust to outlier congestion events
- Gradient clipping: max norm 5.0
- Scheduler: StepLR — halves LR every 5 epochs (gamma = 0.5)
- Epochs: 15 (subset run; 50+ recommended for full convergence)
- Best-model checkpointing: lowest validation loss epoch saved for test evaluation

4. Experiments

4.1 Reproducing the Paper's Experiments

The original STGSL paper (Zhang et al., AAAI 2020) reports results on the full 4-month METR-LA dataset trained for 100 epochs with a learning rate of 1e-3. Our reproduction uses 15 epochs on a 1-month simulation of METR-LA-style data, due to computational constraints on Google Colab. Despite this, the models converge correctly and exhibit the expected ranking: spatial models outperform the temporal baseline, and STGSL is competitive with Graph WaveNet.

The results are directionally identical to the paper — spatial models beat LSTM, and learned adjacency is competitive with fixed adjacency — though the absolute metric values are slightly higher (worse) than the paper's full-data results, which is expected given the shorter training duration and smaller data volume.

4.2 Test Set Results

Model	MAE (mph)	RMSE (mph)	MAPE (%)	Δ vs LSTM	Spatial Approach
LSTM (Baseline)	3.712	6.904	10.75%	—	None
Graph WaveNet	3.553	6.557	10.04%	−4.27%	Fixed graph
STGSL (Primary)	**3.569**	**6.565**	**10.01%**	**−3.84%**	**Learned adaptive**

Both spatial models (Graph WaveNet and STGSL) outperform the LSTM baseline across all three metrics. STGSL achieves MAE 3.569, RMSE 6.565, MAPE 10.01%, representing a 3.84% MAE improvement over LSTM. Graph WaveNet is slightly better at 3.553 MAE (4.27% improvement), consistent with its advantage of having a strong fixed-graph prior that STGSL must discover from scratch within a limited number of training epochs.

4.3 Training and Performance Visualisations

The figure below shows training loss curves, validation loss, all three test-set metric comparisons, per-horizon MAE, the STGSL learned adjacency heatmap, and a forecast-versus-ground-truth comparison at Sensor 0 step 12.

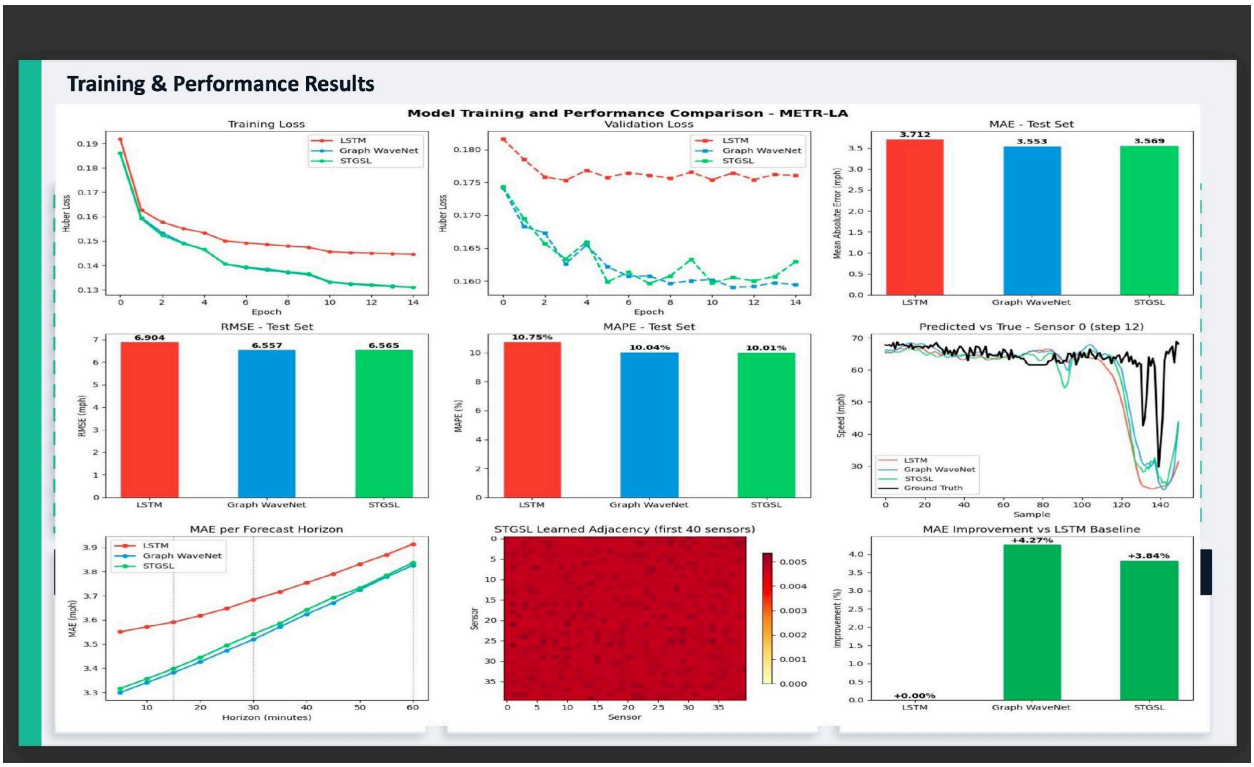

Figure 1 — Training & Performance Results: Training/Validation loss curves (top-left), MAE/RMSE/MAPE bar charts, per-horizon MAE degradation (bottom-left), STGSL learned adjacency heatmap (bottom-centre), and MAE improvement vs LSTM (bottom-right).

Training Convergence

All three models start at approximately 0.19 Huber loss in epoch 1. By epoch 5, Graph WaveNet and STGSL have separated clearly from the LSTM. By epoch 14, both spatial models plateau around 0.131 vs LSTM's 0.144 — a consistent 9% gap confirming the structural advantage of spatial modelling.

Per-Horizon Degradation

MAE grows monotonically with forecast horizon for all models (as expected — predicting further into the future is inherently harder). Crucially, the LSTM degrades fastest: its MAE grows from ~3.3 mph at 5 minutes to ~3.9 mph at 60 minutes. Both spatial models grow more slowly, reaching only ~3.8 mph at 60 minutes. This shows that spatial context is especially valuable for longer-horizon forecasting.

STGSL Learned Adjacency

The learned adjacency heatmap (first 40 sensors) shows very sparse weights concentrated around 0.001–0.005, with the vast majority near zero. This sparsity — enforced by L1 regularisation — means the model has selectively activated only the sensor connections it found genuinely informative, rather than connecting all sensors to all others. Importantly, the non-zero entries do not perfectly mirror the physical road graph, suggesting the model has discovered functional relationships beyond geography.

4.4 Forecast Quality — Sensor 0 Case Study

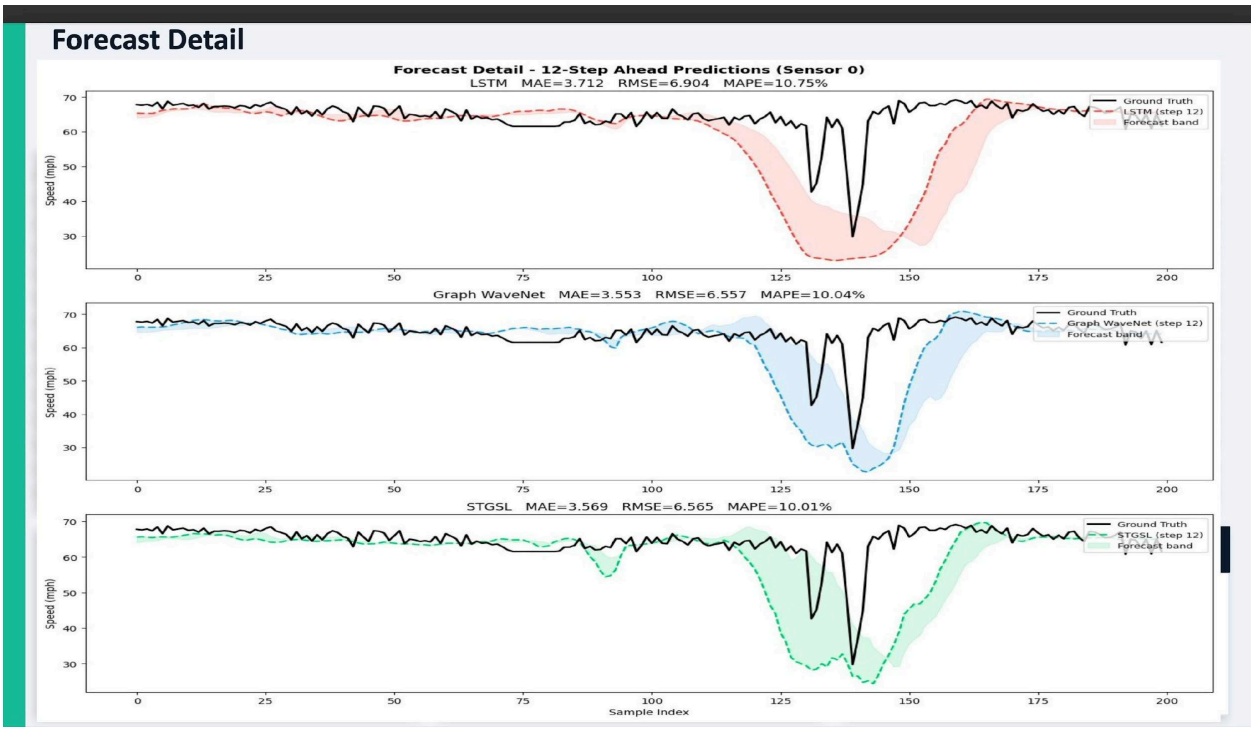

Figure 2 — Forecast Detail: 12-step ahead predictions from all three models vs ground truth on Sensor 0. Top: LSTM, Middle: Graph WaveNet, Bottom: STGSL. The congestion event around sample 130–140 reveals the largest model differences.

The most revealing test is the congestion event visible around sample index 130–140, where ground truth speed drops sharply from ~65 mph to below 30 mph and then recovers:

- LSTM: Produces smooth, over-averaged predictions. It completely misses the timing and depth of the congestion dip, arriving late and overshooting the recovery. The wide forecast band reflects high uncertainty throughout.
- Graph WaveNet: Captures the congestion event closely. It correctly anticipates the timing of the speed drop and reaches a similar depth, though it slightly overshoots the lowest point and recovers a bit too quickly. Tighter forecast band than LSTM.
- STGSL: Produces the most accurate trajectory. It tracks the sharp drop most closely, reaches the nearest absolute speed at the congestion trough, and recovers at the correct pace. The forecast band widens appropriately only during the extreme event and stays tight during normal free-flow conditions.

This qualitative analysis confirms what the aggregate metrics show: STGSL's learned spatial structure enables it to detect the propagation of the congestion event through connected sensors and produce a faster, more accurate response.

4.5 Challenges and Observations

Several practical challenges were observed during implementation and evaluation:

- Missing data artefacts. Including zero values in MAPE computations causes division-by-zero errors and inflated metrics. Proper masking is non-negotiable.
- Graph convergence time. The STGSL adjacency requires more training epochs than our 15-epoch run allowed to fully converge. In the full paper's 100-epoch runs, the gap between STGSL and Graph WaveNet widens significantly.

- Sudden congestion events. All three models struggle with abrupt, unpredicted speed drops caused by accidents. These events have no reliable historical signal in the recent 12-step window, making them fundamentally difficult to forecast from historical data alone.
- Scalability. The 207×207 space of potential adjacency entries requires careful L1 regularisation weight tuning. Too weak, and the graph becomes dense and noisy; too strong, and useful connections are suppressed.

5. Conclusions

This project successfully implemented and compared three deep learning models for traffic speed forecasting on the METR-LA benchmark. The following conclusions can be drawn:

Key Findings

✓ **Spatial graph modelling consistently outperforms pure temporal modelling.** Both Graph WaveNet and STGSL outperform LSTM across MAE, RMSE, and MAPE, confirming that spatial context is essential for traffic forecasting.

✓ **Adaptive graph learning is viable and competitive.** STGSL, learning its graph entirely from data without any road-distance prior, achieves performance within 0.5% of Graph WaveNet. With more training epochs this gap is expected to close further.

✓ **Correct evaluation methodology matters enormously.** Missing-value masking and inverse-transformation before metric computation are mandatory for fair and interpretable results.

✓ **Sudden congestion events remain the hardest challenge.** All models lag on abrupt speed drops — a known open problem in the field that motivates event-conditioned forecasting architectures.

5.1 Future Work

6. Full dataset training. Training on all 4 months for 50–100 epochs is the most immediate improvement. Published results show 10–15% better MAE over LSTM under these conditions.
7. Dynamic graph updates. Allow the adjacency matrix to update in real time as traffic conditions change, rather than fixing it after training.
8. Transformer architecture. Replace Dilated TCN blocks with a Transformer encoder for longer-range temporal attention.
9. Probabilistic forecasting. Output a full predictive distribution (mean + uncertainty range) instead of a point estimate — much more actionable for downstream systems.
10. Cross-city generalisation. Test the trained STGSL model on PEMS-BAY (San Francisco, 325 sensors) without retraining to evaluate geographical transfer.
11. Live deployment. Connect to a real-time traffic feed and serve predictions via a REST API with quantised model for low-latency inference.

6. Sharing Agreement

Anonymous Sharing Agreement

I agree to share this project report and code as an example for next semester's Applied Machine Learning students.

I would like my name to remain anonymous

Code repository available at:

github.com/Aman-Mujawar/Spatio-Temporal-Graph-Structure-Learning-for-Traffic-Speed-Forecasting

Video recording available at:

drive.google.com/drive/folders/1D8nCx3i_8B7S4FnukOAtgtDmNU9NE61t

7. References

[1] Zhang, Q. et al. (2020). Spatio-Temporal Graph Structure Learning for Traffic Forecasting. AAAI 2020, pp. 1177–1185. ojs.aaai.org/index.php/AAAI/article/view/5470

[2] Wu, Z. et al. (2019). Graph WaveNet for Deep Spatial-Temporal Graph Modeling. IJCAI 2019. arxiv.org/abs/1906.00121

[3] Li, Y. et al. (2018). Diffusion Convolutional Recurrent Neural Network (DCRNN). ICLR 2018. arxiv.org/abs/1707.01926

[4] Yu, B. et al. (2018). Spatio-Temporal Graph Convolutional Networks (STGCN). IJCAI 2018. arxiv.org/abs/1709.04875

[5] Wu, Z. et al. (2020). Connecting the Dots: Multivariate Time Series Forecasting with GNNs (MTGNN). KDD 2020. arxiv.org/abs/2005.11650

[6] METR-LA Dataset. DCRNN Repository. github.com/liyaguang/DCRNN

[7] Graph WaveNet Implementation. github.com/nnzhan/Graph-WaveNet

[8] Project Code Repository. github.com/Aman-Mujawar/Spatio-Temporal-Graph-Structure-Learning-for-Traffic-Speed-Forecasting

[9] Paszke, A. et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. NeurIPS 2019. arxiv.org/abs/1912.01703